SEGUNDO DEPARTAMENTAL- EJERCICIO 21- ORDENAMIENTO INDIRECTO (PE)

En este programa desarrollamos un sistema de ordenamiento indirecto utilizando el algoritmo Quick Sort, con el propósito de organizar diferentes tipos de datos sin modificar directamente la posición de los elementos originales. Para ello, trabajamos con números enteros, caracteres y registros de personas, utilizando un arreglo auxiliar de índices que permite establecer el orden de los elementos según su valor, también creamos una estructura llamada Persona, que almacena el nombre y la edad de cada individuo, además, implementamos funciones para inicializar los índices y realizar el ordenamiento de cada tipo de dato mediante llamadas recursivas.

Código Completo:

```
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

// Nuevo tipo de dato
struct Persona {
    string nombre;
    int edad;
};

// Inicializa el arreglo de indices
void inicializarIndices(int indices[], int n) {
    for (int i = 0; i < n; i++) {
        indices[i] = i;
    }
}

// Quick Sort indirecto para enteros
void quickSortEnteros(int datos[], int indices[],
                      int inicio, int fin) {
    int i = inicio, j = fin;
    int pivote = datos[indices[(inicio + fin) / 2]];

    while (i <= j) {
        while (datos[indices[i]] < pivote) i++;
        while (datos[indices[j]] > pivote) j--;

        if (i <= j) {
            int aux = indices[i];
            indices[i] = indices[j];
            indices[j] = aux;
            i++;
            j--;
        }
    }

    if (inicio < j)
        quickSortEnteros(datos, indices, inicio, j);

    if (i < fin)
        quickSortEnteros(datos, indices, i, fin);
}
```

```
// Quick Sort indirecto para caracteres
void quickSortCaracteres(char datos[], int indices[],
                         int inicio, int fin) {
    int i = inicio, j = fin;
    char pivote = datos[indices[(inicio + fin) / 2]];

    while (i <= j) {
        while (datos[indices[i]] < pivote) i++;
        while (datos[indices[j]] > pivote) j--;

        if (i <= j) {
            int aux = indices[i];
            indices[i] = indices[j];
            indices[j] = aux;
            i++;
            j--;
        }
    }

    if (inicio < j)
        quickSortCaracteres(datos, indices, inicio, j);

    if (i < fin)
        quickSortCaracteres(datos, indices, i, fin);
}

// Quick Sort indirecto para personas por edad
void quickSortPersonas(Persona datos[], int indices[],
                       int inicio, int fin) {
    int i = inicio, j = fin;
    int pivote = datos[indices[(inicio + fin) / 2]].edad;

    while (i <= j) {
        while (datos[indices[i]].edad < pivote) i++;
        while (datos[indices[j]].edad > pivote) j--;

        if (i <= j) {
            int aux = indices[i];
            indices[i] = indices[j];
            indices[j] = aux;
            i++;
            j--;
        }
    }

    if (inicio < j)
        quickSortPersonas(datos, indices, inicio, j);

    if (i < fin)
        quickSortPersonas(datos, indices, i, fin);
}

int main() {
    int opcion, n;
    int indices[MAX];

    cout << "=== QUICK SORT INDIRECTO - PE ===\n";
    cout << "1. Enteros\n";
    cout << "2. Caracteres\n";
    cout << "3. Personas por edad\n";
    cout << "Selecciona una opcion: ";
    cin >> opcion;
```

```
    cout << "Cantidad de elementos (1-100): ";
    cin >> n;

    if (n < 1 || n > MAX) {
        cout << "Cantidad no valida.\n";
        return 0;
    }

    if (opcion == 1) {
        int datos[MAX];

        for (int i = 0; i < n; i++) {
            cout << "Entero " << i + 1 << ": ";
            cin >> datos[i];
        }

        inicializarIndices(indices, n);
        quickSortEnteros(datos, indices, 0, n - 1);

        cout << "\nDatos originales: ";
        for (int i = 0; i < n; i++)
            cout << datos[i] << " ";

        cout << "\nDatos ordenados indirectamente: ";
        for (int i = 0; i < n; i++)
            cout << datos[indices[i]] << " ";

        cout << "\nIndices ordenados: ";
        for (int i = 0; i < n; i++)
            cout << indices[i] << " ";
    }
    else if (opcion == 2) {
        char datos[MAX];

        for (int i = 0; i < n; i++) {
            cout << "Caracter " << i + 1 << ": ";
            cin >> datos[i];
        }

        inicializarIndices(indices, n);
        quickSortCaracteres(datos, indices, 0, n - 1);

        cout << "\nDatos originales: ";
        for (int i = 0; i < n; i++)
            cout << datos[i] << " ";

        cout << "\nDatos ordenados indirectamente: ";
        for (int i = 0; i < n; i++)
            cout << datos[indices[i]] << " ";

        cout << "\nIndices ordenados: ";
        for (int i = 0; i < n; i++)
            cout << indices[i] << " ";
    }
    else if (opcion == 3) {
        Persona datos[MAX];

        for (int i = 0; i < n; i++) {
            cout << "Nombre: ";
            cin >> datos[i].nombre;
            cout << "Edad: ";
            cin >> datos[i].edad;
        }
```

```
        inicializarIndices(indices, n);
        quickSortPersonas(datos, indices, 0, n - 1);

        cout << "\nPersonas originales:\n";
        for (int i = 0; i < n; i++)
            cout << datos[i].nombre << " - "
                 << datos[i].edad << " anios\n";

        cout << "\nPersonas ordenadas por edad:\n";
        for (int i = 0; i < n; i++) {
            int pos = indices[i];
            cout << datos[pos].nombre << " - "
                 << datos[pos].edad << " anios\n";
        }

        cout << "Indices ordenados: ";
        for (int i = 0; i < n; i++)
            cout << indices[i] << " ";
    }
    else {
        cout << "Opcion no valida.\n";
    }

    return 0;
}
```

Explicación del Código:

1. Declaración de la Constante MAX (const int MAX = 100;)

Declaramos una constante llamada MAX con valor de cien, que establece la cantidad máxima de elementos que podemos almacenar en los arreglos del programa. Utilizamos una constante para mantener un límite definido y facilitar su modificación en caso de que sea necesario.

2. Definición de la Estructura Persona (struct Persona)

Creamos una estructura llamada Persona, que funciona como un nuevo tipo de dato definido por el usuario. Su objetivo es agrupar la información de cada individuo dentro de un mismo registro, permitiendo almacenar su nombre y edad.

3. Función para Inicializar los Índices (void inicializarIndices(int indices[], int n))

Creamos una función encargada de inicializar el arreglo de índices que se utilizará durante el ordenamiento indirecto. Recibe dos parámetros: indices[], que representa el arreglo auxiliar, y n, que indica la cantidad de elementos que se van a procesar. Mediante un ciclo for, asignamos a cada posición su propio número de índice, comenzando desde cero.

4. Función Quick Sort Indirecto para Enteros (void quickSortEnteros(int datos[], int indices[], int inicio, int fin))

Es la primera función de ordenamiento y se encarga de organizar indirectamente un arreglo de números enteros. Recibe cuatro parámetros: el arreglo original datos[], el

arreglo auxiliar indices[], la posición inicial inicio y la posición final fin, a diferencia del Quick Sort tradicional, esta función no intercambia los números directamente, sino que modifica las posiciones almacenadas en el arreglo de índices, de esta manera, los datos originales permanecen en sus posiciones iniciales, mientras que el arreglo auxiliar establece el nuevo orden.

5. Función Quick Sort Indirecto para Caracteres (void quickSortCaracteres(char datos[], int indices[], int inicio, int fin))

Es la segunda función de ordenamiento y realiza el mismo procedimiento que la función anterior, pero trabaja con caracteres. Recibe el arreglo original de caracteres, el arreglo auxiliar de índices y los límites de la sección que se desea ordenar. selecciona un carácter central como pivote, compara los demás caracteres con él y realiza los intercambios únicamente dentro del arreglo de índices y finalmente, utiliza llamadas recursivas para completar el ordenamiento sin modificar las posiciones originales de los caracteres.

6. Función Quick Sort Indirecto para Personas (void quickSortPersonas(Persona datos[], int indices[], int inicio, int fin))

Es la tercera función de ordenamiento y permite organizar indirectamente un arreglo de personas tomando como referencia su edad. Recibe el arreglo original de estructuras Persona, el arreglo auxiliar de índices y los límites de la sección que se desea ordenar, las comparaciones se realizan utilizando el atributo edad de cada persona y cuando es necesario cambiar de posición dos elementos, solamente intercambiamos sus índices, conservando intactos los registros originales.

7. Función Principal (int main())

Es el punto de entrada del programa. Dentro de ella declaramos las variables necesarias para controlar el funcionamiento del sistema, incluyendo la opción seleccionada, la cantidad de elementos y el arreglo auxiliar de índices, también contiene el menú de opciones, la validación de los datos y los procedimientos para capturar, ordenar y mostrar la información.

8. Ordenamiento Indirecto de Números Enteros (if (opcion == 1))

Cuando el usuario selecciona la primera opción, declaramos un arreglo llamado datos de tipo int, con capacidad para cien números, después utilizamos un ciclo for para solicitar los números y almacenarlos en el arreglo original. Una vez capturados todos los elementos, llamamos a inicializarIndices(indices, n) para establecer las posiciones originales de cada número y después, ejecutamos quickSortEnteros(datos, indices, 0, n - 1) para ordenar indirectamente los elementos.

9. Ordenamiento Indirecto de Caracteres (else if (opcion == 2))

Cuando el usuario selecciona la segunda opción, declaramos un arreglo llamado datos de tipo char, con capacidad para cien caracteres. Mediante un ciclo for solicitamos los caracteres y los almacenamos en sus respectivas posiciones, después, llamamos a inicializarIndices(indices, n) para establecer las posiciones originales y ejecutamos quickSortCaracteres(datos, indices, 0, n - 1) para realizar el ordenamiento indirecto.

Finalmente, mostramos los caracteres originales, los caracteres ordenados mediante el arreglo de índices y las posiciones originales que corresponden al nuevo orden.

10. Ordenamiento Indirecto de Personas por Edad (else if (opcion == 3))

Cuando el usuario selecciona la tercera opción, declaramos un arreglo llamado datos de tipo Persona, con capacidad para cien registros. Mediante un ciclo for solicitamos el nombre y la edad de cada individuo y almacenamos la información en sus respectivos atributos, después, llamamos a inicializarIndices(indices, n) para establecer las posiciones originales y ejecutamos quickSortPersonas(datos, indices, 0, n - 1) para ordenar indirectamente a las personas según su edad.

SEGUNDO DEPARTAMENTAL- EJERCICIO 21- ORDENAMIENTO INDIRECTO (POO)

En este programa desarrollamos un sistema de ordenamiento indirecto utilizando el algoritmo Quick Sort, con el propósito de organizar diferentes tipos de datos sin modificar directamente la posición de los elementos originales. Para ello, trabajamos con números enteros, caracteres y registros de personas, utilizando un arreglo auxiliar de índices que permite establecer el orden de los elementos según su valor, también creamos una estructura llamada Persona, que almacena el nombre y la edad de cada individuo, además, implementamos funciones para inicializar los índices y realizar el ordenamiento de cada tipo de dato mediante llamadas recursivas.

Código Completo:

```
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

// Clase que representa el nuevo tipo de dato
class Persona {
public:
    string nombre;
    int edad;

    Persona() {
        nombre = "";
        edad = 0;
    }

    Persona(string n, int e) {
        nombre = n;
        edad = e;
    }

    void mostrar() {
        cout << nombre << " - " << edad << " anios\n";
    }
};

// Clase que realiza el ordenamiento indirecto
class Ordenador {
private:
    int indices[MAX];

public:
    void inicializarIndices(int n) {
        for (int i = 0; i < n; i++)
            indices[i] = i;
    }

    // Ordenamiento indirecto de enteros
    void quickSort(int datos[], int inicio, int fin) {
        int i = inicio, j = fin;
        int pivote = datos[indices[(inicio + fin) / 2]];

        while (i <= j) {
            while (datos[indices[i]] < pivote) i++;
```

```
            while (datos[indices[j]] > pivote) j--;

            if (i <= j) {
                int aux = indices[i];
                indices[i] = indices[j];
                indices[j] = aux;
                i++;
                j--;
            }
        }

        if (inicio < j) quickSort(datos, inicio, j);
        if (i < fin) quickSort(datos, i, fin);
    }

    // Ordenamiento indirecto de caracteres
    void quickSort(char datos[], int inicio, int fin) {
        int i = inicio, j = fin;
        char pivote = datos[indices[(inicio + fin) / 2]];

        while (i <= j) {
            while (datos[indices[i]] < pivote) i++;
            while (datos[indices[j]] > pivote) j--;

            if (i <= j) {
                int aux = indices[i];
                indices[i] = indices[j];
                indices[j] = aux;
                i++;
                j--;
            }
        }

        if (inicio < j) quickSort(datos, inicio, j);
        if (i < fin) quickSort(datos, i, fin);
    }

    // Ordenamiento indirecto de personas por edad
    void quickSort(Persona datos[], int inicio, int fin) {
        int i = inicio, j = fin;
        int pivote = datos[indices[(inicio + fin) / 2]].edad;

        while (i <= j) {
            while (datos[indices[i]].edad < pivote) i++;
            while (datos[indices[j]].edad > pivote) j--;

            if (i <= j) {
                int aux = indices[i];
                indices[i] = indices[j];
                indices[j] = aux;
                i++;
                j--;
            }
        }

        if (inicio < j) quickSort(datos, inicio, j);
        if (i < fin) quickSort(datos, i, fin);
    }

    // Consultar un indice ordenado
    int obtenerIndice(int posicion) {
        return indices[posicion];
    }
```

```
};

int main() {
    Ordenador ordenador;
    int opcion, n;

    cout << "=== QUICK SORT INDIRECTO - POO ===\n";
    cout << "1. Enteros\n";
    cout << "2. Caracteres\n";
    cout << "3. Personas por edad\n";
    cout << "Selecciona una opcion: ";
    cin >> opcion;

    cout << "Cantidad de elementos (1-100): ";
    cin >> n;

    if (n < 1 || n > MAX) {
        cout << "Cantidad no valida.\n";
        return 0;
    }

    ordenador.inicializarIndices(n);

    if (opcion == 1) {
        int datos[MAX];

        for (int i = 0; i < n; i++) {
            cout << "Entero " << i + 1 << ": ";
            cin >> datos[i];
        }

        ordenador.quickSort(datos, 0, n - 1);

        cout << "\nDatos originales: ";
        for (int i = 0; i < n; i++)
            cout << datos[i] << " ";

        cout << "\nDatos ordenados: ";
        for (int i = 0; i < n; i++)
            cout << datos[ordenador.obtenerIndice(i)] << " ";

        cout << "\nIndices ordenados: ";
        for (int i = 0; i < n; i++)
            cout << ordenador.obtenerIndice(i) << " ";
    }
    else if (opcion == 2) {
        char datos[MAX];

        for (int i = 0; i < n; i++) {
            cout << "Caracter " << i + 1 << ": ";
            cin >> datos[i];
        }

        ordenador.quickSort(datos, 0, n - 1);

        cout << "\nDatos originales: ";
        for (int i = 0; i < n; i++)
            cout << datos[i] << " ";

        cout << "\nDatos ordenados: ";
        for (int i = 0; i < n; i++)
            cout << datos[ordenador.obtenerIndice(i)] << " ";
```

```
        cout << "\nIndices ordenados: ";
        for (int i = 0; i < n; i++)
            cout << ordenador.obtenerIndice(i) << " ";
    }
    else if (opcion == 3) {
        Persona datos[MAX];

        for (int i = 0; i < n; i++) {
            string nombre;
            int edad;

            cout << "Nombre: ";
            cin >> nombre;
            cout << "Edad: ";
            cin >> edad;

            datos[i] = Persona(nombre, edad);
        }

        ordenador.quickSort(datos, 0, n - 1);

        cout << "\nPersonas originales:\n";
        for (int i = 0; i < n; i++)
            datos[i].mostrar();

        cout << "\nPersonas ordenadas por edad:\n";
        for (int i = 0; i < n; i++) {
            int pos = ordenador.obtenerIndice(i);
            datos[pos].mostrar();
        }

        cout << "Indices ordenados: ";
        for (int i = 0; i < n; i++)
            cout << ordenador.obtenerIndice(i) << " ";
    }
    else {
        cout << "Opcion no valida.\n";
    }

    return 0;
}
```

Explicación del Código:

1. Declaración de la Constante Máxima (const int MAX = 100)

Esta constante establece el límite máximo de elementos que pueden almacenarse en los arreglos del programa. Se utiliza para controlar la cantidad de información que puede ingresar el usuario y evitar que se exceda el espacio disponible.

2. Declaración de la Clase Persona (class Persona)

Se define la clase Persona, que representa un nuevo tipo de dato y permite almacenar información de cada persona.

3. Declaración del Constructor Predeterminado (Persona())

Se define un constructor sin parámetros que inicializa los atributos de la clase Persona.

4. Declaración del Constructor con Parámetros (Persona(string n, int e))

Se define un segundo constructor que recibe dos parámetros: un nombre y una edad, su función es asignar directamente los valores recibidos a los atributos del objeto.

Este constructor permite crear objetos con información específica desde el momento de su inicialización.

5. Declaración del Método Mostrar (void mostrar())

Se define el método mostrar(), cuya función es presentar en pantalla el nombre y la edad de una persona.

6. Declaración de la Clase Ordenador (class Ordenador)

Se define la clase Ordenador, que contiene los métodos encargados de realizar el ordenamiento indirecto mediante Quick Sort. Esta clase permite organizar diferentes tipos de datos utilizando un mismo arreglo de índices, sin modificar directamente los arreglos originales, dentro de ella se declara un atributo privado y varios métodos públicos que permiten inicializar los índices, ordenar los datos y consultar las posiciones resultantes.

7. Ordenamiento Indirecto de Números Enteros (quickSort(int datos[], int inicio, int fin))

Se declara el primer método quickSort(), encargado de ordenar indirectamente un arreglo de números enteros.

8. Sobrecarga del Método Quick Sort para Caracteres (quickSort(char datos[], int inicio, int fin))

Se declara un segundo método llamado quickSort(), encargado de ordenar indirectamente un arreglo de caracteres. Esto permite comparar caracteres de acuerdo con sus valores en la codificación utilizada por el sistema.

9. Sobrecarga del Método Quick Sort para Personas (quickSort(Persona datos[], int inicio, int fin))

Se declara un tercer método llamado quickSort(), encargado de ordenar indirectamente objetos de tipo Persona. Este método recibe un arreglo de objetos Persona, además de las posiciones inicial y final del segmento que se va a ordenar, adiferencia de los métodos anteriores, el ordenamiento se realiza utilizando el atributo edad de cada objeto.

10. Función Principal (int main())

Se declara la función principal del programa, que controla su ejecución. Dentro de esta función se crea un objeto llamado ordenador, perteneciente a la clase Ordenador, a partir de esta función se solicitan los datos, se ejecuta el método correspondiente y se muestran los resultados.